

Sajilo Rescue – Event Driven Microservices Backend Flow

Sajilo Rescue is a real-time emergency response system designed using a decoupled, event-driven microservices architecture. The goal is to minimize emergency response time, ensure system resilience, and demonstrate production-level backend engineering skills.

1. High-Level Architecture

The system follows an Event-Driven Architecture. Every critical action such as emergency creation, ambulance matching, voice upload, and acceptance is emitted as an event. This ensures that slow operations like media uploads or database writes do not block emergency dispatch.

2. Technology Stack

- Backend Framework: NestJS with Microservices module
- Real-time Communication: Socket.IO (WebSockets)
- Message Broker: RabbitMQ or NATS
- Primary Database: PostgreSQL
- Caching & Geo-Spatial Search: Redis (Redis GEO)
- Push Notifications: Firebase Cloud Messaging (FCM)
- Media Storage: AWS S3 or Cloudinary

3. Microservices Breakdown

Gateway Service

Acts as the single entry point for all mobile and web clients. Handles JWT authentication, WebSocket connections, and routes emergency requests to the Dispatch Service.

Locator Service

Responsible for tracking all active ambulances using Redis GEO. Ambulances send location heartbeats every 3–5 seconds. The service performs spatial queries to find the nearest available ambulances.

Dispatch & Notification Service

This is the system brain. It manages the entire emergency lifecycle including incident creation, ambulance selection, accept/reject logic, siren notifications, and state transitions.

Media Service

Handles patient voice recordings generated through long-press actions. Audio files are uploaded to cloud storage and an event is emitted containing the emergency case ID and audio URL.

Admin Dashboard Service

Provides administrators with a real-time overview of all emergencies. Admins can monitor live ambulance movement, listen to patient voice messages, and manually override assignments when required.

4. Complete Backend Flow

- Users authenticate and their socket IDs and FCM tokens are stored in Redis.
- Ambulances continuously send GPS heartbeats to the Locator Service.
- Patient triggers emergency by pressing the call button with location data.
- Dispatcher creates a PENDING emergency record in PostgreSQL.
- Dispatcher requests nearest ambulances from Locator Service.
- Top three ambulances receive high-priority siren push notifications.

- First ambulance to accept acquires Redis lock and wins the case.
- Dispatcher marks incident as ACTIVE and cancels others.
- Patient and ambulance sockets are linked for real-time tracking.
- Admin dashboard displays live emergency and audio playback.
- Ambulance completes handover and incident is archived.

5. Engineering Highlights

Concurrency is handled using Redis distributed locks to prevent multiple ambulances from accepting the same emergency. Performance is optimized using Redis GEO spatial searches that complete within milliseconds. System resilience is achieved through message brokers ensuring no emergency alert is lost.